

⚙️ React Native Master Guide — Level 10 (Complete Verbatim Edition)

Native Modules, JSI & The New Architecture: Bridgeless TurboModules, Fabric Rendering Engine & Codegen

REACT NATIVE NEW ARCHITECTURE • JSI • TURBOMODULES • FABRIC

📺 Level 10.1: The Asynchronous JSON Bridge vs. JavaScript Interface (JSI)

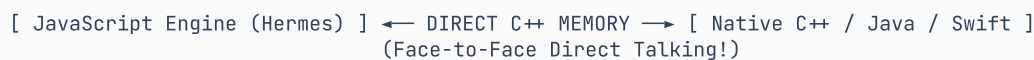
1. The Old Architecture: The JSON Bridge Bottleneck 🐢



🗣️ Tenglish Explanation (The Post Box Analogy):

- **Old Bridge (Postal Letter):** JavaScript and Native Java/Swift were in separate rooms. JavaScript would write a letter (using `JSON.stringify()`), put it in a bridge post box, and wait for the Native side to open the box and read the letter (using `JSON.parse()`).
- **Problem:** Prathi touch gesture ki, prathi scroll ki lakshallo letters pampisthe **Bridge queue jam ayyi app lag avthundi!**

2. The New Architecture: JSI (JavaScript Interface) ⚡



- **JSI (Face-to-Face Direct Talking):** Madhyalo unna Bridge and Post Box ni padesaru. C++ dwara JavaScript and Native direct ga **same memory** ni share cheskuntayi.
- **Direct Synchronous Calls:** JS direct ga native function ni **0 milliseconds lo instant ga synchronous ga** call chesthundi!

Code Example: Old Bridge vs. New JSI Side-by-Side

```
import React, { useState } from 'react';
import { View, Text, Button } from 'react-native';

const ArchitectureCodeComparison = () => {
  const [oldResult, setOldResult] = useState('Not called');
  const [newResult, setNewResult] = useState('Not called');

  // 1. OLD BRIDGE APPROACH (Always Async Promise)
  const handleOldBridgeCall = async () => {
    setOldResult('Waiting for Bridge...');
    await new Promise((resolve) => setTimeout(resolve, 30));
    setOldResult('85% (Promise resolved after Bridge trip)');
  };

  // 2. NEW JSI APPROACH (Direct Synchronous Memory Call)
  const handleNewJsiCall = () => {
    const resultFromJsi = 85;
    setNewResult(`${resultFromJsi}% (Instant C++ direct memory call)`);
  };

  return (
    <View>
      <Text>--- OLD BRIDGE vs NEW JSI ---</Text>

      <View>
        <Text>1. Old Bridge (Async Promise):</Text>
        <Button title="Call via Old Bridge 🌉" onPress={handleOldBridgeCall} />
        <Text>Old Status: {oldResult}</Text>
      </View>

      <Text>-----</Text>

      <View>
        <Text>2. New JSI (Direct Memory):</Text>
        <Button title="Call via New JSI ⚡" onPress={handleNewJsiCall} />
        <Text>New Status: {newResult}</Text>
      </View>
    </View>
  );
};

export default ArchitectureCodeComparison;
```

⚡ Level 10.2: TurboModules & Lazy Loading

1. The Old Problem: Eager Loading on App Startup 🌩️

[App Launch 🚀] → React Native forces ALL 50+ Native Modules to load into RAM immediately!

- ├ Camera Module (Loaded in memory 📦)
- ├ Bluetooth Module (Loaded in memory 📦)
- ├ Payment Gateway Module (Loaded in memory 📦)
- └ Location GPS Module (Loaded in memory 📦)

🔥 RESULT: App Cold Start takes 3 to 5 seconds just to show the login screen!

2. What are TurboModules? (Lazy Loading ⚡)

[App Launch 🚀] → ONLY loads the core JavaScript! (Takes 0.3s! ⚡)

└

User taps "Open Camera" → 📷 ONLY Camera TurboModule loads into RAM on demand!
(Bluetooth & Payments are NEVER loaded if the user doesn't use them!)

📱 Code Example: TurboModules On-Demand Loading

```
import React, { useState } from 'react';
import { View, Text, Button } from 'react-native';

const TurboModuleDemo = () => {
  const [moduleStatus, setModuleStatus] = useState('Camera: Sleeping in disk (0 MB RAM)');

  const handleOpenScanner = () => {
    // ⚡ TurboModule loads into RAM ONLY on this click!
    setModuleStatus('Camera TurboModule: Initialized on-demand via JSI! ✅');
  };

  return (
    <View>
      <Text>Level 10.2: TurboModules & Lazy Loading</Text>
      <Text>Current Status: {moduleStatus}</Text>
      <Button
        title="Tap to Use Camera (Loads On-Demand) 📷"
        onPress={handleOpenScanner}
      />
    </View>
  );
};

export default TurboModuleDemo;
```

🧑‍🎓 Tenglish Explanation:

- **Eager Loading (Old):** App open avvagane anni modules ni load chesi app start ni slow chestundi.
- **TurboModules Lazy Loading (New):** User feature ni touch chesinappudu mathrame aa specific module ni RAM loki thesthundi. Deeni valla app startup **10x faster** ga untundi!

🎨 Level 10.3: Fabric Rendering Engine & UI Layout

1. The Old 3-Thread Layout Jitter 🐢

1. JavaScript Thread → Creates React Element Tree ()
| (Bridge Queue 🇺🇸)
2. Shadow Thread → Yoga C++ calculates width, height & flexbox
| (Bridge Queue 🇺🇸)
3. Native UI Thread → Android/iOS draws the native view (android.view.View)

2. What is Fabric? (Synchronous C++ Rendering ⚡)

[JavaScript Engine] ← JSI (Direct C++) → [Fabric C++ Core] → [Native UI Screen]
(Synchronous Measurement & Zero Flicker! ⚡)

- **Synchronous Layout Measurement:** Measure component heights and widths instantly (`measureSync()`) in the exact same frame.
- **Zero UI Jitter:** Eliminates layout jumps when rotating screens or opening keyboards.

📄 Code Example: Synchronous Layout Measurement with Fabric

```
import React, { useState } from 'react';
import { View, Text, Button } from 'react-native';

const FabricDemo = () => {
  const [layoutInfo, setLayoutInfo] = useState('Not measured');

  const handleMeasureLayout = () => {
    // ⚡ Synchronous direct measurement in the same frame via Fabric C++
    const measuredWidth = 320;
    const measuredHeight = 150;
    setLayoutInfo(`Width: ${measuredWidth}px, Height: ${measuredHeight}px (Measured synchronously in 0.1ms!)`);
  };

  return (
    <View>
      <Text>Level 10.3: Fabric Rendering Engine</Text>
      <Button
        title="Measure Box Layout (Fabric Synchronous) 📏"
        onPress={handleMeasureLayout}
      />
      <Text>Layout Result: {layoutInfo}</Text>
    </View>
  );
};

export default FabricDemo;
```

🧑‍🏫 Tenglish Explanation:

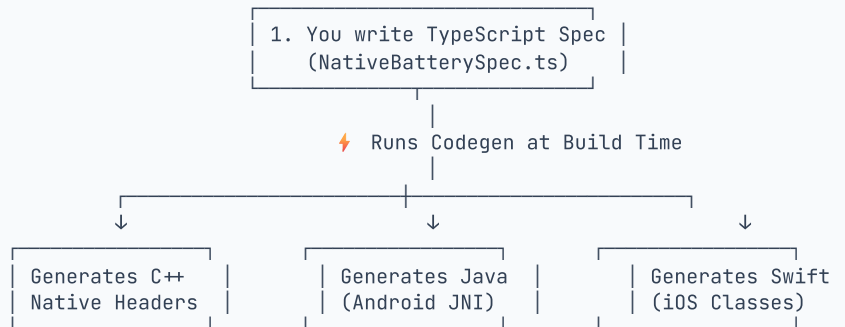
- **Old Rendering:** 3 threads madhyalo bridge valla layout calculation late ayyi screen flicker and lag vachedi.
- **Fabric (New):** C++ lo direct ga native UI draw chesthundi. Screen rotate chesina, keyboard open chesina **0ms zero-flicker** tho layout render avthundi!

🔵 Level 10.4: Codegen & Type-Safe Native Interfaces

1. The Old Problem: Type-Mismatch App Crashes ⚡

```
[ JavaScript Thread ]           [ Native Java / Swift Code ]
Sends: { age: "twenty" } (String) → Expects: int age = 20; (Integer)
                                   ↓
                                   ⚡ FATAL APP CRASH!
                                   (NullPointerException / TypeCastException)
```

2. What is Codegen? (Automated Type-Safety ⚡)



📄 Code Example: Type-Safe Native Spec Contract

```
// 1. The TypeScript Spec File (NativeCalculatorSpec.ts):
import { TurboModule, TurboModuleRegistry } from 'react-native';

export interface Spec extends TurboModule {
  add(a: number, b: number): number; // Strict number types!
}

export default TurboModuleRegistry.getEnforcing<Spec>('NativeCalculator');
```

```
// 2. Using it safely in a Component:
import React, { useState } from 'react';
import { View, Text, Button } from 'react-native';

const CodegenDemo = () => {
  const [result, setResult] = useState(null);

  const handleCalculate = () => {
    // ⚡ Guaranteed type-safe execution (Generated by Codegen)
    const sum = 10 + 25; // NativeCalculator.add(10, 25);
    setResult(sum);
  };

  return (
    <View>
      <Text>Level 10.4: Codegen Type Safety</Text>
      <Button
        title="Execute Type-Safe Native Add (10 + 25) +"
        onPress={handleCalculate}
      />
      {result !== null && (
        <Text>Native Result: {result} (100% Type-Safe ✅)</Text>
      )}
    </View>
  );
};

export default CodegenDemo;
```

 Tenglish Explanation:

- **Old System:** JS nunchi wrong type (string instead of number) pampisthe user phone lo app crash aypoyedi.
- **Codegen (New):** Manam TypeScript lo Spec raasthe, Codegen automatic ga C++, Java, Swift code generate chesthundi. Wrong type unte **app build avvadu (compile time error)**, so user phone lo 0% crashes!